

256-Point FFT Implementation

Ahmed Hussien Ashour

- ▶ Introduction
- ▶ System Modeling.
- ▶ RTL Design.
- ▶ Verification.
- ▶ ASIC Flow.

Introduction

- ▶ Radix: radix-2
- ▶ Decimation Type: Decimation in Frequency (DIF)
- ▶ Architecture: Single Delay Feedback (SDF)
- ▶ Size: 256-Point
- ▶ Data Format: Fixed-Point
- ▶ I/O Order: Natural Order Input , Bit-Reversed Output

How Does the SDF implementation realize the DFT equation?

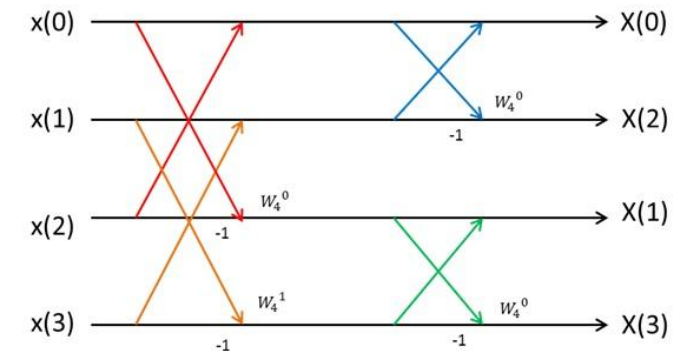
- DFT equation:

$$X[k] = \sum_{n=0}^{N-1} x[n] \times e^{-j2\pi \frac{kn}{N}}, k = 0, 1 \dots N-1$$

- For Simplicity if we had a 4-point FFT the DFT equation should be:

$$X[k] = \sum_{n=0}^3 x[n] \times e^{-j2\pi \frac{kn}{4}}, k = 0, 1, 2, 3$$

- $X[0] = x[0] + x[1] + x[2] + x[3]$
- $X[1] = x[0] - j \cdot x[1] - x[2] + j \cdot x[3]$
- $X[2] = x[0] - x[1] + x[2] - x[3]$
- $X[3] = x[0] + j \cdot x[1] - x[2] - j \cdot x[3]$

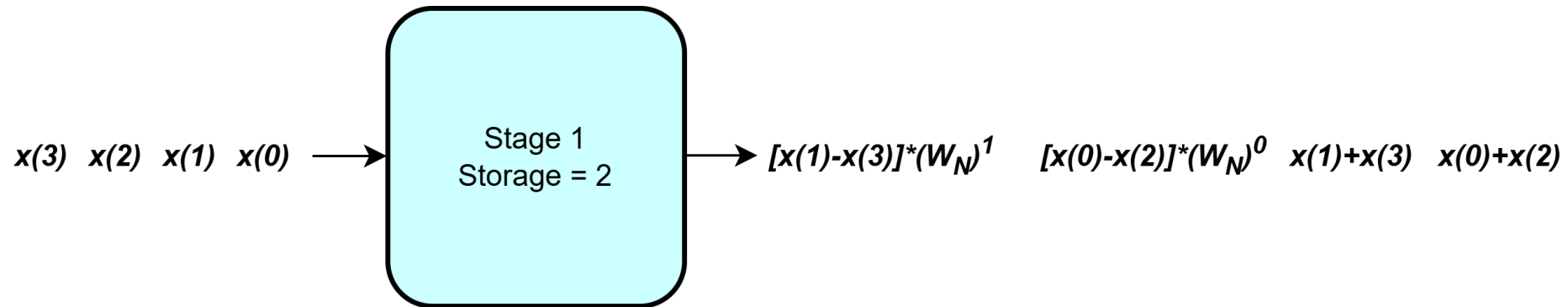


How Does the SDF implementation realize the DFT equation?

First SDF stage receives the data serially in $x(0)$, $x(1)$, $x(2)$ and then $x(3)$

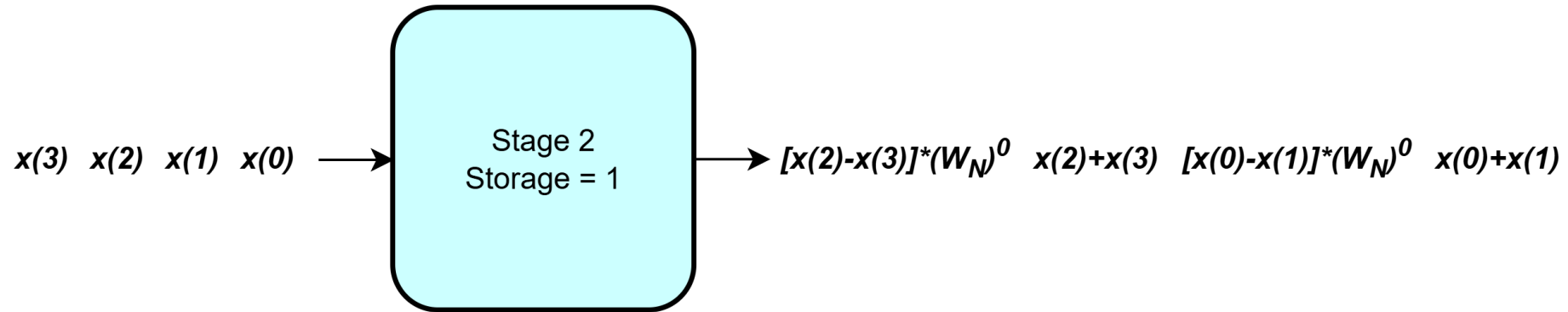
The operation is divided into 2 stages:

- ▶ Storing new data and outputting old data: the first half of the input samples [$x(0)$ and $x(1)$] is getting stored (while the subtraction of the previous block is passing the multiplier stage and transferred to the next stage)
- ▶ Calculating the sum and difference and outputting the sum terms of this block, the first output of this block is $x(0)+x(2)$ and then $x(1)+x(3)$ and subtraction terms is then stored.
- ▶ Storing new data and outputting old data, the subtraction terms of the previous block is passing the multiplier stage and transferred to the next stage which is $[x(0) - x(2)] \cdot W_4^0$ and $[x(1) - x(3)] \cdot W_4^1$ while first half of the new input is getting stored



How Does the SDF implementation realize the DFT equation?

- ▶ The second stage does the same operation but with half the storage capacity, so it does it two times.

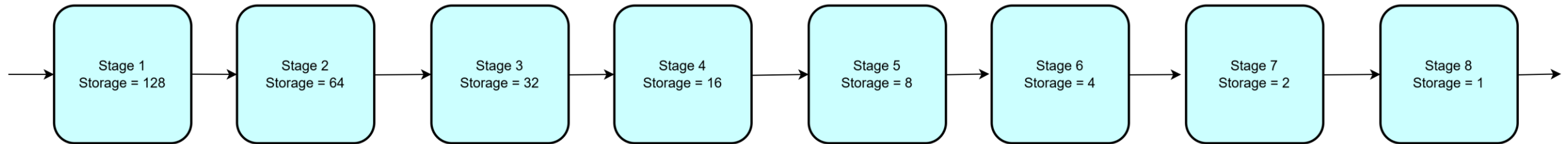


- ▶ When the output of the first stage is fed into the input of the second stage, the final output is the dft equation but after undergoing bit reversal.
- ▶ By Sub. $W_4^0 = e^{-j2\pi\frac{0}{4}} = 1$ and $W_4^1 = e^{-j2\pi\frac{1}{4}} = -j$

$$\begin{aligned}
 x(0) &\rightarrow x(0) + x(2) && \rightarrow x(0) + x(1) + x(2) + x(3) && \rightarrow X(0) \\
 x(1) &\rightarrow x(1) + x(3) && \rightarrow x(0) - x(1) + x(2) - x(3) && \rightarrow X(2) \\
 x(2) &\rightarrow [x(0) - x(2)] && \rightarrow x(0) - jx(1) - x(2) + jx(3) && \rightarrow X(1) \\
 x(3) &\rightarrow [-jx(1) + jx(3)] && \rightarrow x(0) + jx(1) - x(2) - jx(3) && \rightarrow X(3)
 \end{aligned}$$

How Does the SDF implementation realize the DFT equation?

- Now It can be scaled to 256-Point by cascading stages with double the storage capacity



- The Inputs is assumed to be complex (16-bit real and 16-bit imaginary) natural order
- The output is going to be the (16-bit real and 16-bit imaginary) bit reversed
- The latency is expected to be $(128+64+32+16+8+4+2+1) = 255$ cycle

System Modeling

- ▶ A MATLAB model of the algorithm was developed to match the stages operations and randomize different inputs to use the output to compare the SDF model to the MATLAB's `fft()` function and calculate the SQNR of the model.
- ▶ The random values of the (inputs, signals, outputs) was used to decide the number of bits used to represent the integer parts of the signals.
- ▶ The fraction parts of the signals was decided based on the SQNR of the model.
- ▶ Different data types were used as the inputs to the model (Double, Single, Fixed Point).
- ▶ Finally, the test dumps the inputs & outputs of the model to a text file (for verification) and the twiddle factors to be stored in the ROMs

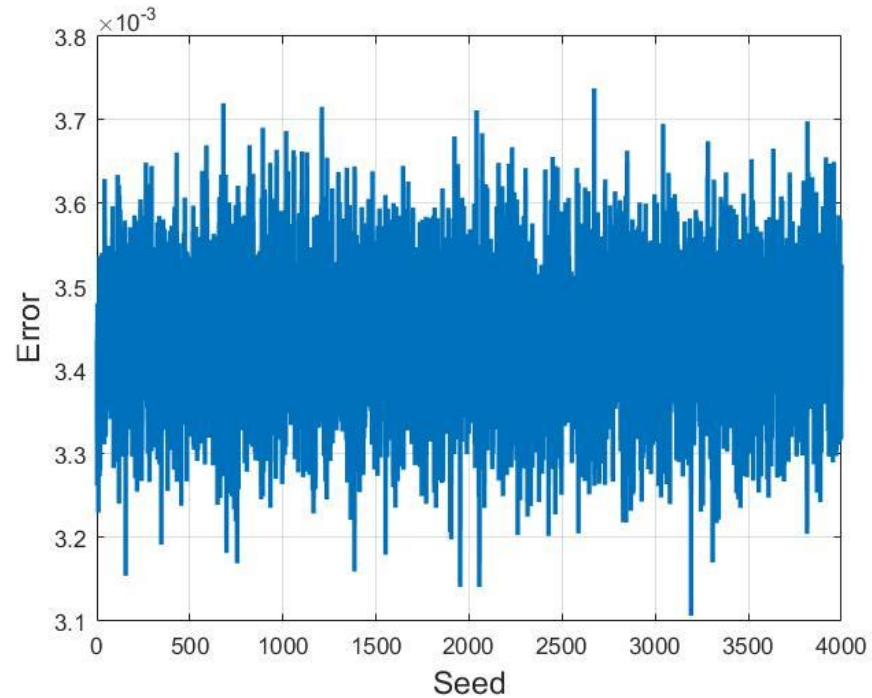
► The results of the Instrumentation:

y	Output	256 × 1	complex double	Signed	32	24	No	-74.43869936513619	76.1177154560931
x	Input	256 × 1	complex double	Signed	32	28	No	-5.138779575988432	4.952259317033658
stg_1	Local	256 × 1	complex double	Signed	32	28	No	-7.030736991485153	6.901697867765938
stg_2	Local	256 × 1	complex double	Signed	32	27	No	-9.960689036638263	9.677228831171009
stg_3	Local	256 × 1	complex double	Signed	32	27	No	-14.563822818423022	13.89051614118424
stg_4	Local	256 × 1	complex double	Signed	32	26	No	-18.986508898797503	21.07249311929346
stg_5	Local	256 × 1	complex double	Signed	32	26	No	-26.34287539494023	27.820042300566953
stg_6	Local	256 × 1	complex double	Signed	32	25	No	-39.74079620284324	41.73719561696291
stg_7	Local	256 × 1	complex double	Signed	32	25	No	-58.45294623535585	56.31046385840634
stg_8	Local	256 × 1	complex double	Signed	32	24	No	-74.43869936513619	76.1177154560931
idxVec	Local	128 × 1	double	Signed	32	0	Yes	0	127
twTable	Local	128 × 1	complex double	Signed	32	30	No	-1	1
twTable_double	Local	128 × 1	complex double	Signed	32	30	No	-1	1
► T	Input	1 × 1	struct 	-	-	-	No	-	-
i	Local	1 × 1	double	Signed	32	0	Yes	1	256
n	Local	1 × 1	double	Signed	32	0	Yes	0	127
N	Local	1 × 1	double	Signed	32	0	Yes	256	256
p	Local	1 × 1	double	Signed	32	0	Yes	0	127
Wp	Local	1 × 1	complex double	Signed	32	30	No	-1	1

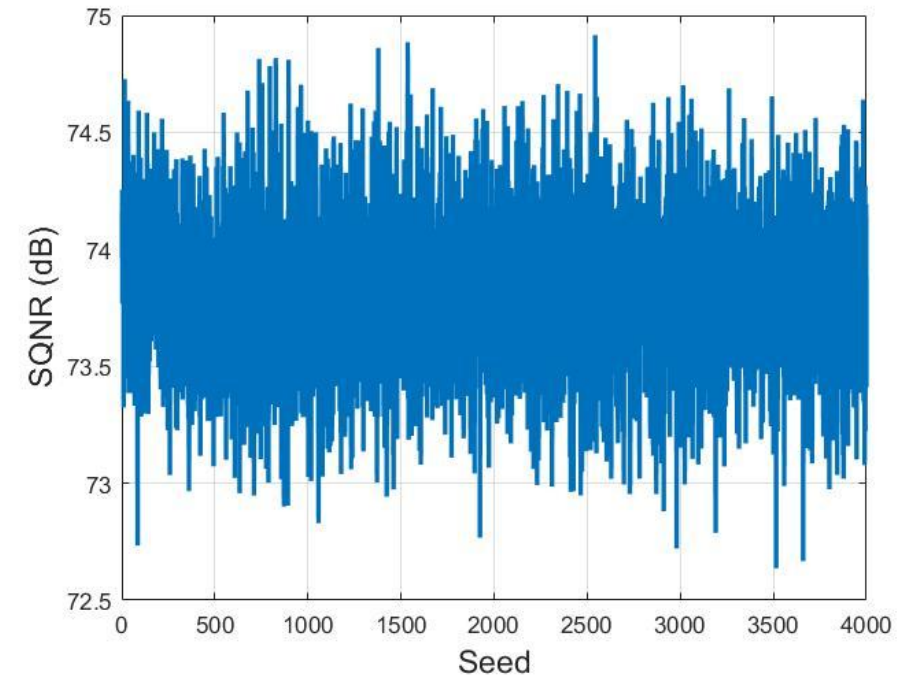
- Using the results to determine the number of bits to represent the integer and finding the fraction parts of the signals based on the SQNR of the model:

```
case 'FxpT'  
    %using a custom fimath to match the rtl truncating of bits instead  
    %of default rounding to nearest  
    fm = fimath('RoundingMethod','Floor');  
    T.x    = fi([], 1, 4+12, 12, fm);  
    T.WN   = fi([], 1, 2+22, 22, fm);  
    T.stg1 = fi([], 1, 4+20, 20, fm);  
    T.stg2 = fi([], 1, 5+19, 19, fm);  
    T.stg3 = fi([], 1, 5+19, 19, fm);  
    T.stg4 = fi([], 1, 6+18, 18, fm);  
    T.stg5 = fi([], 1, 6+18, 18, fm);  
    T.stg6 = fi([], 1, 7+17, 17, fm);  
    T.stg7 = fi([], 1, 7+17, 17, fm);  
    T.stg8 = fi([], 1, 8+16, 16, fm);  
    T.y    = fi([], 1, 8+8, 8, fm);
```

- The results of using the previous bit values:



average error = 3.439029e-03

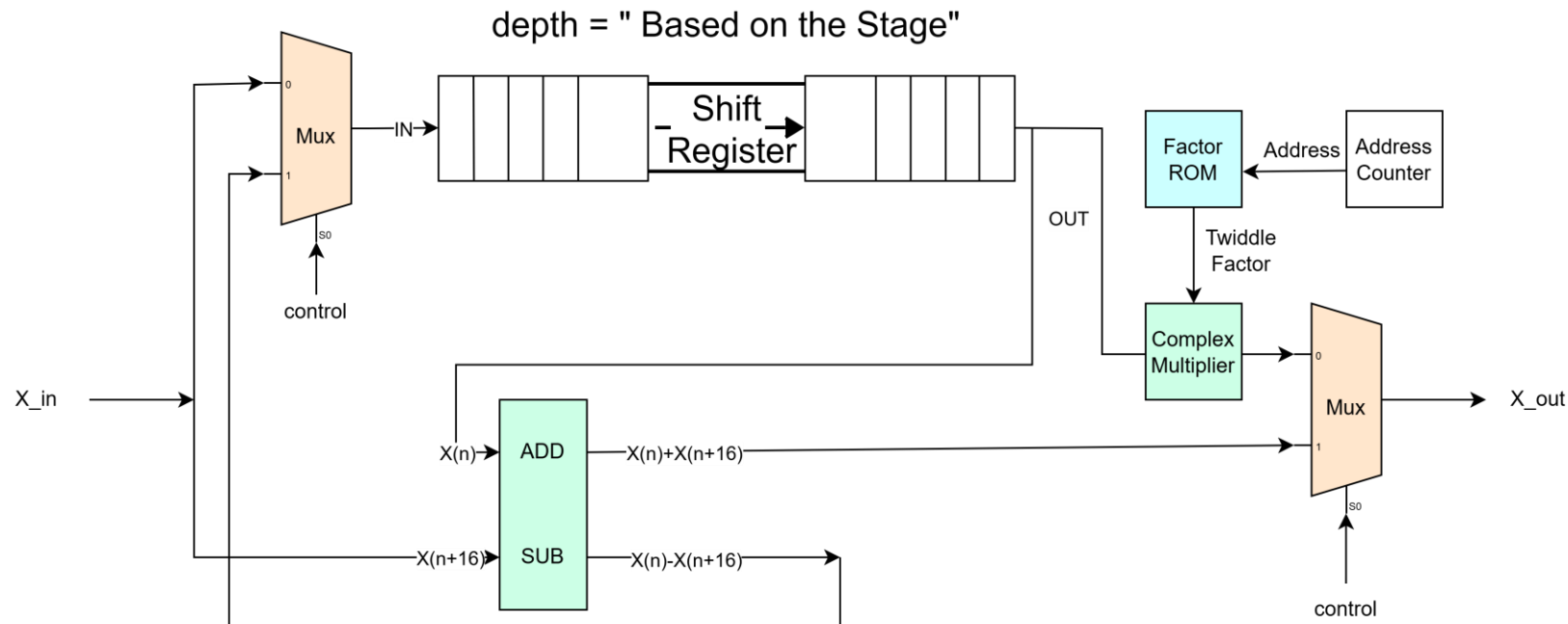


average sqnr = 73.83 dB

RTL Design

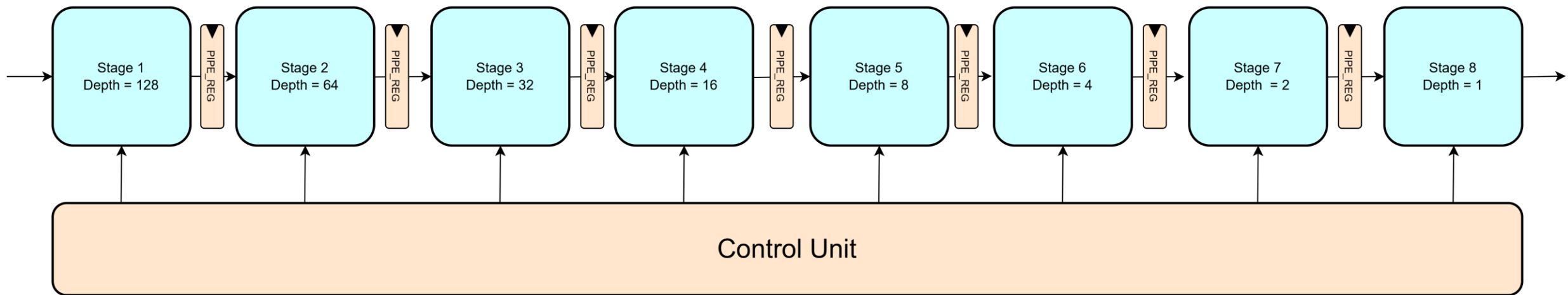
Operation:

1. Storing new data and outputting old data, control=0, the first half of the input samples is getting stored in the shift register (while the subtraction terms of the previous block is passing the multiplier stage and transferred to the next stage)
2. Calculating the sum and difference and outputting the addition of this block, control=1, now the first output of this stage is the sum terms and while the subtracted terms is stored in the shift register.
3. Storing new data and outputting old data, control=0, the first half of the new input is getting stored in the shift register (while the subtraction terms of the previous block is passing through the multiplier stage and transferred to the next stage)



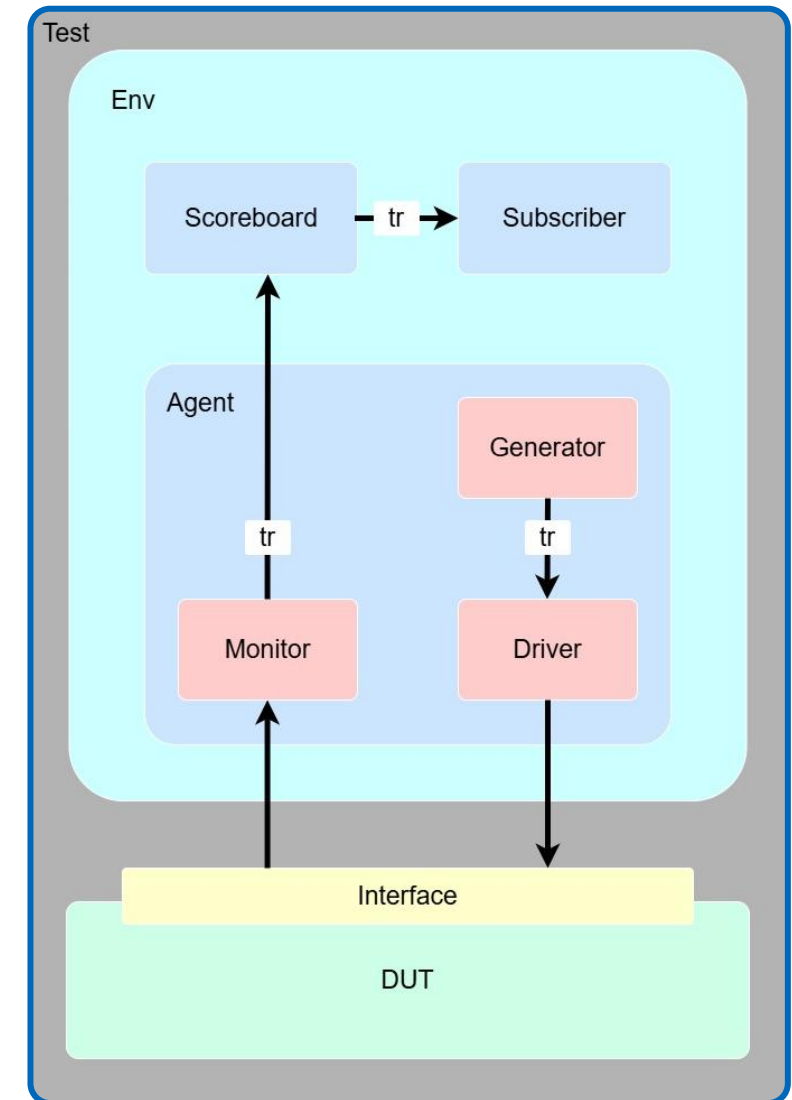
Full System Block Diagram

- ▶ Assembling the full system:
- ▶ The Control Unit is just a counter with its bits connected as controls to the stages in reverse order so that the first stage toggles every 128 cycle (corresponding to the 128 sample) and so on.
- ▶ Pipeline registers are inserted between each stage to reduce the delay of the critical path
- ▶ This makes the Latency of the system $255 + 7 (\text{Pipeline}) = 262$ cycle



Verification

- ▶ The generator reads the MATLAB's Inputs file and prepares a transaction and sends it to the Driver
- ▶ The driver receives the transaction and drives the DUT through the clocking block a little bit after the positive clock edge.
- ▶ The monitor reads the Interface through its clocking block a little bit before the positive clock edge and sends the captured values to the scoreboard.
- ▶ The scoreboard send the received data to the subscriber for coverage and then if the DUT's valid flag is asserted the DUT's output is compared with the MATLAB's golden output and the result is recorded



Functional Coverage

```
TYPE /test_sv_unit/subscriber/cg_dout      100.00%    100    -    Covered
covered/total bins:                       10         10    -
missing/total bins:                       0         10    -
% Hit:                                    100.00%    100    -
Coverpoint cp_dout_real                    100.00%    100    -    Covered
covered/total bins:                       5         5     -
missing/total bins:                       0         5     -
% Hit:                                    100.00%    100    -
bin neg_full                             34         1     -    Covered
bin neg_mid                             512983      1     -    Covered
bin zero                                102         1     -    Covered
bin pos_mid                             510850      1     -    Covered
bin pos_full                             33         1     -    Covered
Coverpoint cp_dout_imag                    100.00%    100    -    Covered
covered/total bins:                       5         5     -
missing/total bins:                       0         5     -
% Hit:                                    100.00%    100    -
bin neg_full                             29         1     -    Covered
bin neg_mid                             511803      1     -    Covered
bin zero                                105         1     -    Covered
bin pos_mid                             512032      1     -    Covered
bin pos_full                             33         1     -    Covered
```

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 4

Total Coverage By Instance (filtered view): 100.00%

Code Coverage

```
=====
=== Design Unit: work.fft_256_top
=====
Enabled Coverage      Bins      Hits      Misses  Coverage
-----
Branches              8         8         0    100.00%
Conditions            2         2         0    100.00%
Expressions           2         2         0    100.00%
Statements            11        11         0    100.00%
Toggles              10852     10644      208    98.08%
```

Total Coverage By Design Unit (filtered view): 99.61%

Simulator Transcript

```
# scoreboard: passed tests: 1024000, failed tests: 0
# subscriber: valid_count=1024000
```

- The Toggle Coverage is not 100% because the last stages ROMs has basic twiddle values like 1 and -j which results in less bit toggling since they are represented in 24 bits

ASIC Flow

- ▶ Technology:

- Skywater 130nm (sky130A PDK)

- ▶ Cells:

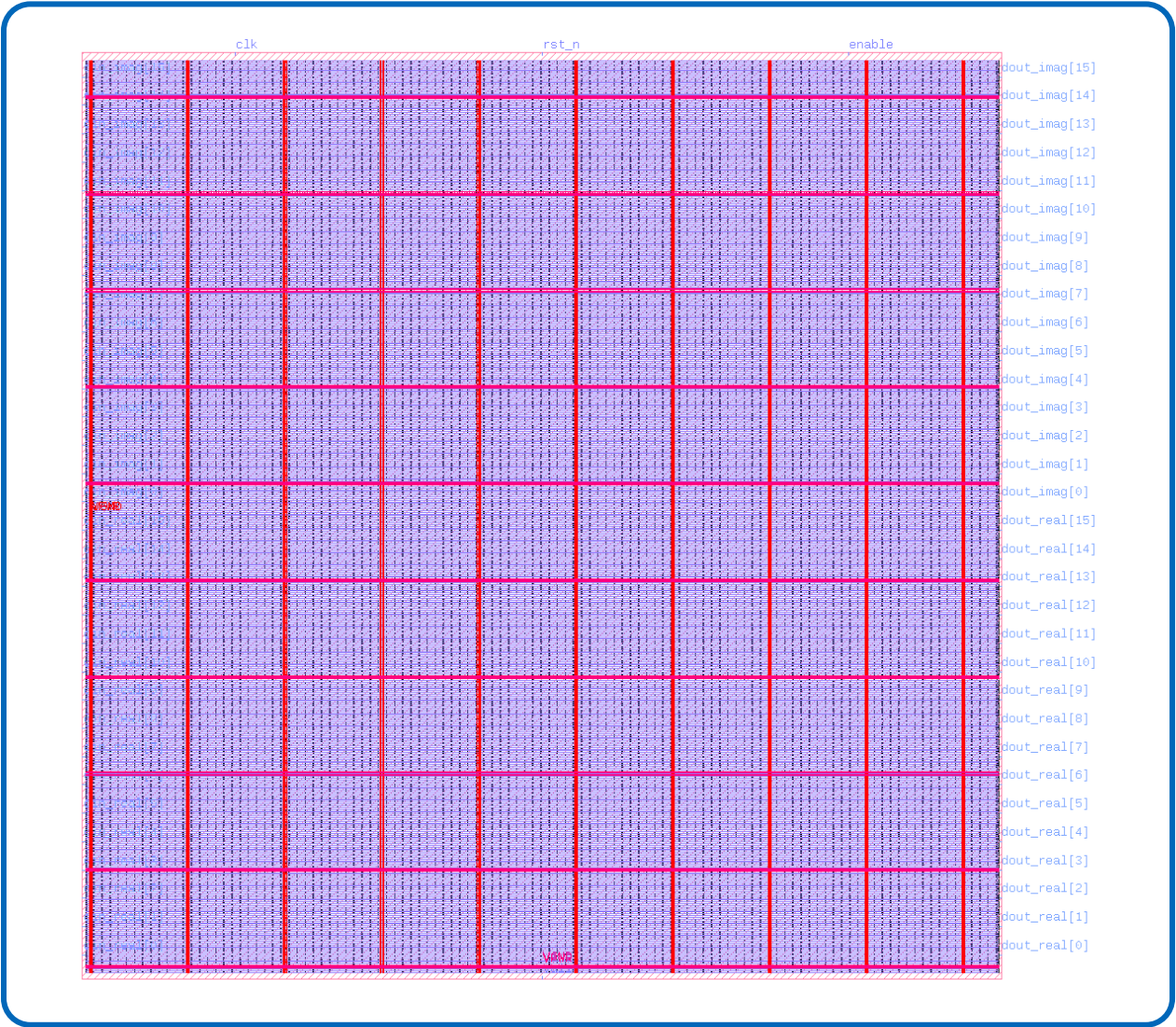
- The sky130_fd_sc_hd cells were used for their high density in the area-optimized run

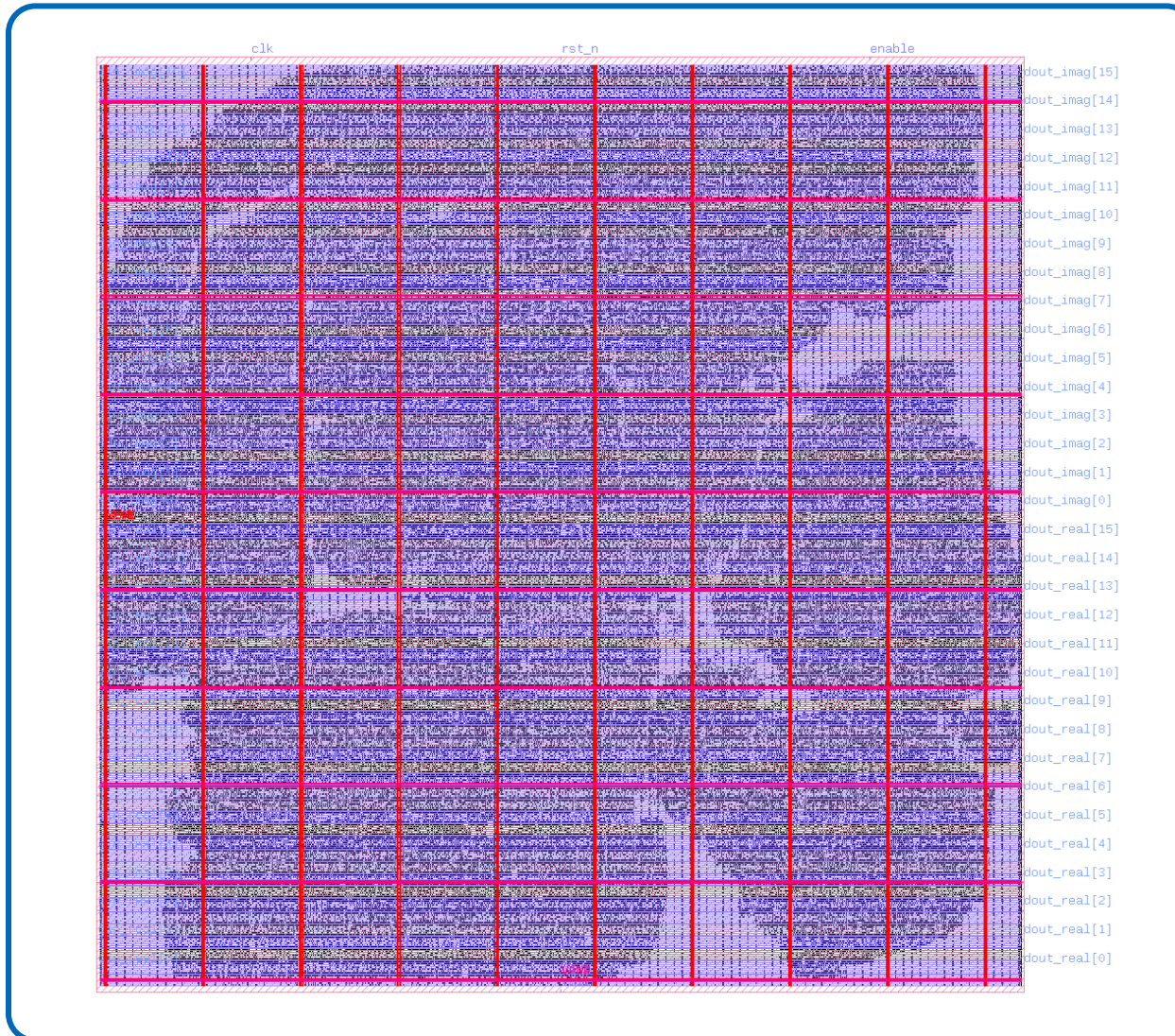
- ▶ Setup & Hold Uncertainty of 100ps and 200ps were added to the configuration file
- ▶ Clock Period = 17ns
- ▶ Synthesis strategy was AREA 0

```
=== fft_256_top ===
```

Number of wires:	91814
Number of wire bits:	103409
Number of public wires:	339
Number of public wire bits:	11878
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	103054
\$_ANDNOT_	27605
\$_AND_	1071
\$_MUX_	12775
\$_NAND_	2201
\$_NOR_	8954
\$_NOT_	2304
\$_ORNOT_	2184
\$_OR_	8983
\$_XNOR_	8993
\$_XOR_	17179
sky130_fd_sc_hd__dfrrtp_2	819
sky130_fd_sc_hd__dfxtp_2	9986

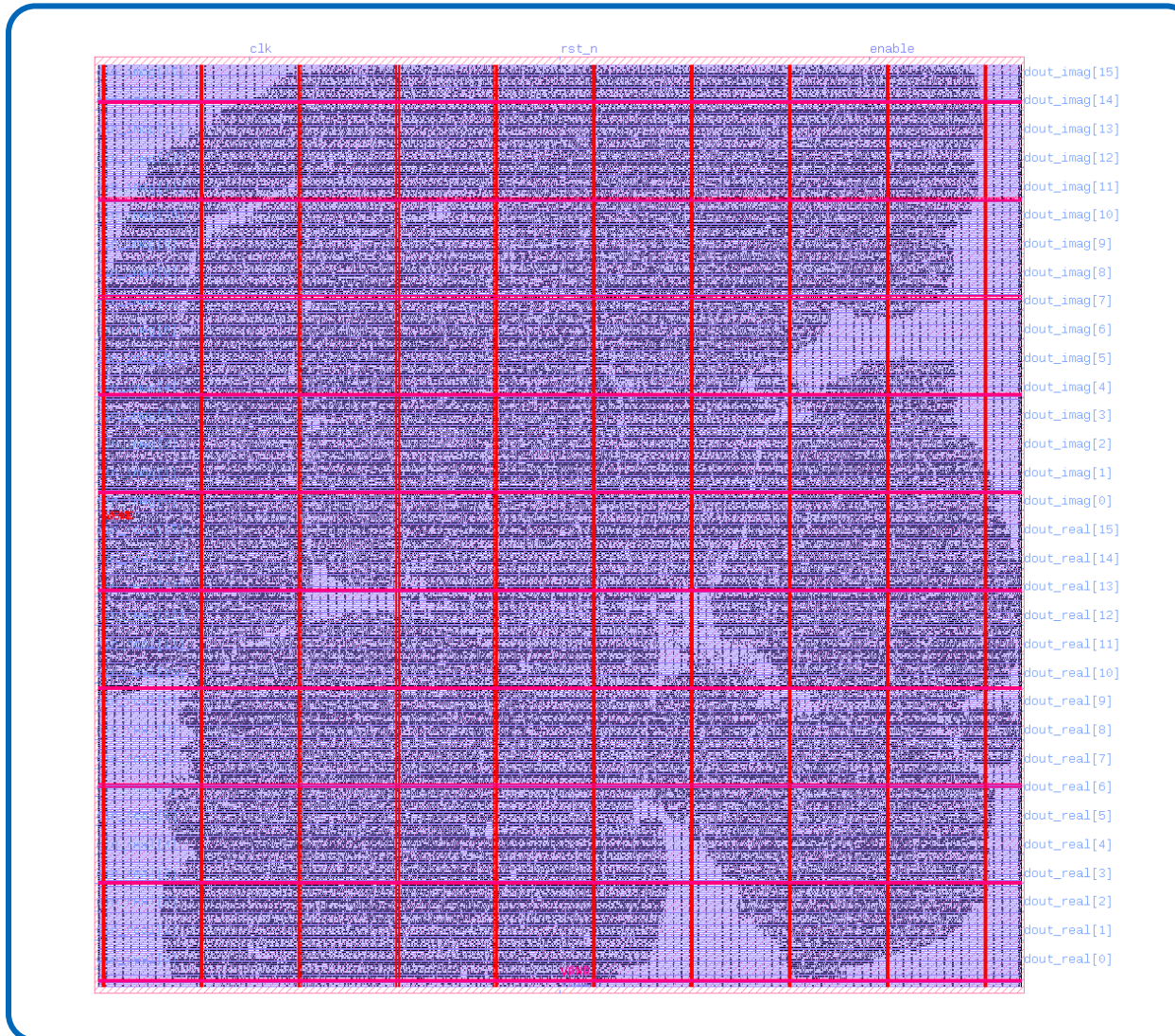
```
Chip area for module '\fft_256_top': 1046384.816001
```



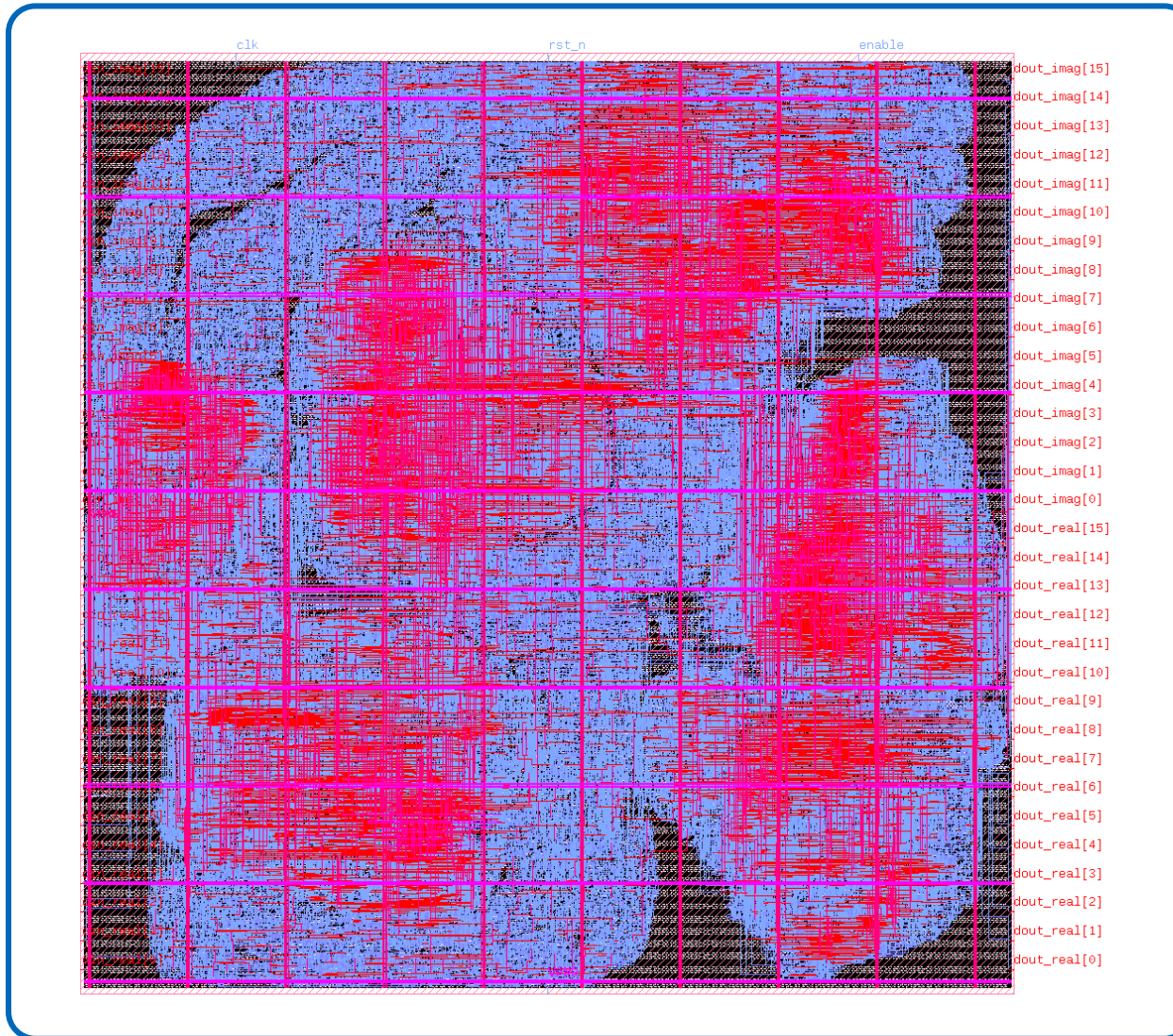


	OUTLINE 1/0
	li1.LABEL 2/1
	li1.PIN 2/2
	li1.OBS 2/3
	met1 3/0
	met1 3/0
	met1.LABEL 3/1
	met1.PIN 3/2
	met1.OBS 3/3
	met2 4/0
	met2 4/0
	met2.LABEL 4/1
	met2.PIN 4/2
	met3 5/0
	met3 5/0
	met3.LABEL 5/1
	met3.PIN 5/2
	met4 6/0
	met4 6/0
	met4.LABEL 6/1
	met4.PIN 6/2
	met5 7/0
	met5 7/0
	met5.LABEL 7/1
	met5.PIN 7/2
	nwell.LABEL 8/1
	nwell.PIN 8/2
	pwell.LABEL 9/1
	pwell.PIN 9/2
	pwell.OBS 9/3
	via 10/0
	via2 11/0
	via3 12/0
	via4 13/0

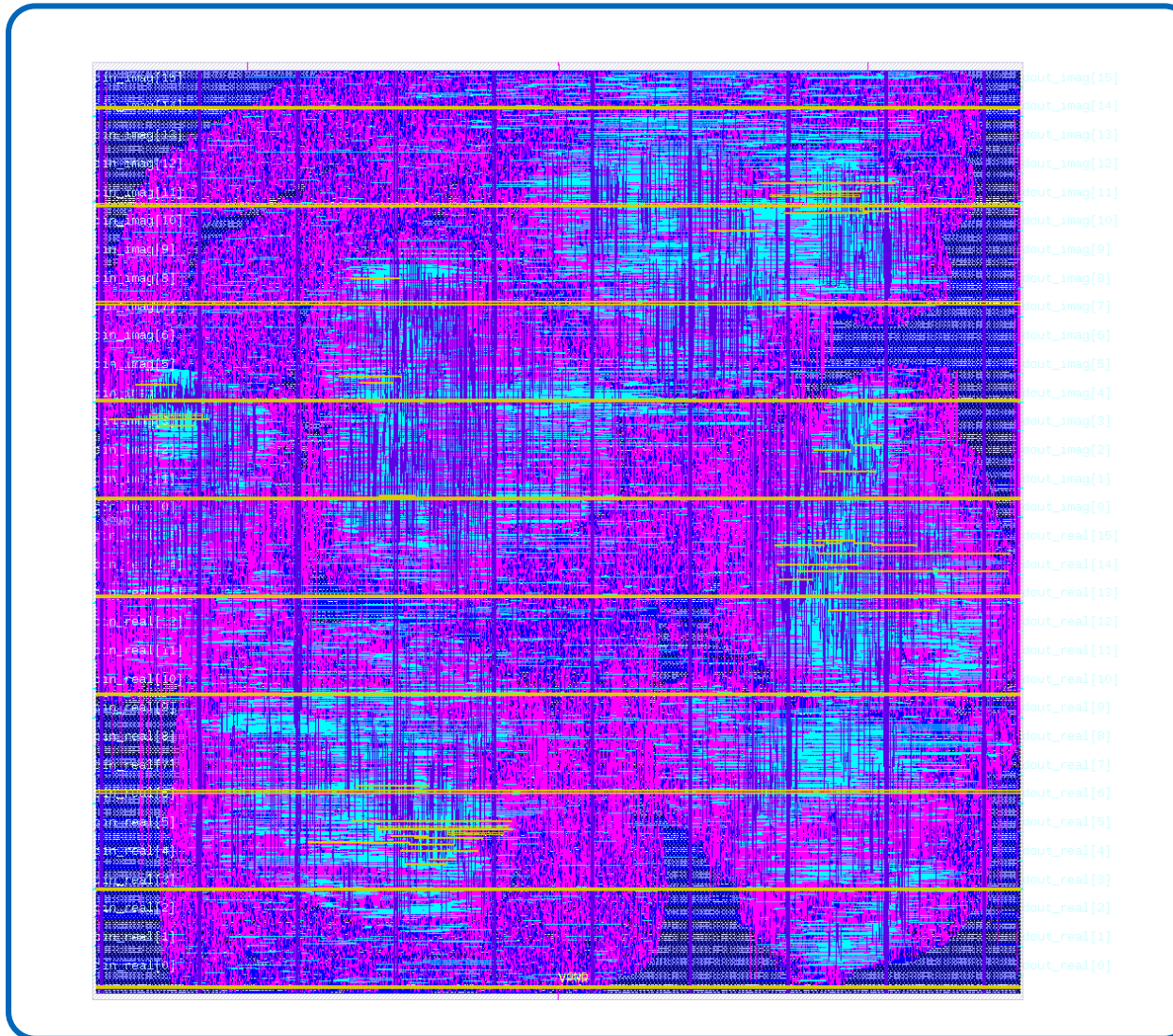
Clock Tree Synthesis



	OUTLINE 1/0
	li1.LABEL 2/1
	li1.PIN 2/2
	li1.OBS 2/3
	met1 3/0
	met1 3/0
	met1.LABEL 3/1
	met1.PIN 3/2
	met1.OBS 3/3
	met2 4/0
	met2 4/0
	met2.LABEL 4/1
	met2.PIN 4/2
	met3 5/0
	met3 5/0
	met3.LABEL 5/1
	met3.PIN 5/2
	met4 6/0
	met4 6/0
	met4.LABEL 6/1
	met4.PIN 6/2
	met5 7/0
	met5 7/0
	met5.LABEL 7/1
	met5.PIN 7/2
	nwell.LABEL 8/1
	nwell.PIN 8/2
	pwell.LABEL 9/1
	pwell.PIN 9/2
	pwell.OBS 9/3
	via 10/0
	via2 11/0
	via3 12/0
	via4 13/0



	OUTLINE 1/0
	li1.LABEL 2/1
	li1.PIN 2/2
	li1.OBS 2/3
	mcon.OBS 3/3
	met1 4/0
	met1 4/0
	met1 4/0
	met1.LABEL 4/1
	met1.PIN 4/2
	met1.OBS 4/3
	met2 5/0
	met2 5/0
	met2 5/0
	met2.LABEL 5/1
	met2.PIN 5/2
	met3 6/0
	met3 6/0
	met3 6/0
	met3.LABEL 6/1
	met3.PIN 6/2
	met4 7/0
	met4 7/0
	met4 7/0
	met4.LABEL 7/1
	met4.PIN 7/2
	met5 8/0
	met5 8/0
	met5 8/0
	met5.LABEL 8/1
	met5.PIN 8/2
	nwell.LABEL 9/1
	nwell.PIN 9/2
	pwell.LABEL 10/1
	pwell.PIN 10/2
	pwell.OBS 10/3
	via 11/0
	via2 12/0
	via3 13/0
	via4 14/0



	prBoundary.boundary - 235/4
	pwell.pin - 122/16
	pwell.label - 64/59
	nwell.drawing - 64/20
	nwell.pin - 64/16
	nwell.label - 64/5
	diff.drawing - 65/20
	tap.drawing - 65/44
	psdm.drawing - 94/20
	nsdm.drawing - 93/44
	poly.drawing - 66/20
	hvtp.drawing - 78/44
	licon1.drawing - 66/44
	npc.drawing - 95/20
	li1.drawing - 67/20
	li1.pin - 67/16
	li1.label - 67/5
	mcon.drawing - 67/44
	met1.drawing - 68/20
	met1.pin - 68/16
	met1.label - 68/5
	via.drawing - 68/44
	met2.drawing - 69/20
	met2.pin - 69/16
	met2.label - 69/5
	via2.drawing - 69/44
	met3.drawing - 70/20
	met3.pin - 70/16
	met3.label - 70/5
	via3.drawing - 70/44
	met4.drawing - 71/20
	met4.pin - 71/16
	met4.label - 71/5
	via4.drawing - 71/44
	met5.drawing - 72/20
	met5.pin - 72/16
	met5.label - 72/5
	areaid.diode - 81/23
	areaid.standarddc - 81/4
	text.drawing - 83/44
	boundary - 236/0

- ▶ The $F_{MAX} = \frac{1}{T_{clk-WNS}}$
- ▶ $T_{clk} = 17ns$
- ▶ $F_{MAX} = 60.2MHz$

```
=====
report_tns
=====
tns 0.00

=====
report_wns
=====
wns 0.00

=====
report_worst_slack -max (Setup)
=====
worst slack 0.39

=====
report_worst_slack -min (Hold)
=====
worst slack 0.30
```

=====					
report_power					
=====					
===== Typical Corner =====					
Group		Internal	Switching	Leakage	Total
		Power	Power	Power	Power (Watts)

Sequential		3.71e-02	6.23e-03	9.35e-08	4.33e-02 2.9%
Combinational		5.94e-01	8.44e-01	5.39e-07	1.44e+00 97.1%
Macro		0.00e+00	0.00e+00	0.00e+00	0.00e+00 0.0%
Pad		0.00e+00	0.00e+00	0.00e+00	0.00e+00 0.0%

Total		6.31e-01	8.50e-01	6.33e-07	1.48e+00 100.0%
		42.6%	57.4%	0.0%	

- ▶ Final Area = $1446.24 \mu m \times 1444.32 \mu m$ With 51.1% utilization
- ▶ NO DRC or LVS violations

```
Design Name: fft_256_top
Run Directory: /openlane/designs/fft_256_top/runs/final_run
-----

Magic DRC Summary:
Source: /openlane/designs/fft_256_top/runs/final_run/reports/signoff/drc.rpt
Total Magic DRC violations is 0
-----

LVS Summary:
Source: /openlane/designs/fft_256_top/runs/final_run/logs/signoff/39-fft_256_top.lef.lvs.log
Number of nets: 116890          |Number of nets: 116890
Design is LVS clean.
-----
```

Thank You!